

WEEK-10-Practical

2520080033

TASK-1

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
root@Shamikahasini:~# echo "This is an inode test file." > original.txt
root@Shamikahasini:~# ls -li original.txt
5636 original.txt
root@Shamikahasini:~# stat original.txt
  File: original.txt
  Size: 28          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 5636          Links: 1
Access: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-10-02 16:22:12.524061628 +0000
Modify: 2026-10-02 16:22:12.524061628 +0000
Change: 2026-10-02 16:22:12.524061628 +0000
 Birth: 2026-10-02 16:22:12.524061628 +0000
root@Shamikahasini:~# ln original.txt hardlink.txt
root@Shamikahasini:~# ls -li original.txt hardlink.txt
5636 -rw-r--r-- 2 root root 28 Oct 2 16:22 hardlink.txt
5636 -rw-r--r-- 2 root root 28 Oct 2 16:22 original.txt
root@Shamikahasini:~# stat original.txt
  File: original.txt
  Size: 28          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 5636          Links: 2
Access: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-10-02 16:22:12.524061628 +0000
Modify: 2026-10-02 16:22:12.524061628 +0000
Change: 2026-10-02 16:22:28.882562988 +0000
 Birth: 2026-10-02 16:22:12.524061628 +0000
root@Shamikahasini:~# ln -s original.txt symlink.txt
root@Shamikahasini:~# ls -li original.txt hardlink.txt symlink.txt
5636 -rw-r--r-- 2 root root 28 Oct 2 16:22 hardlink.txt
5636 -rw-r--r-- 2 root root 28 Oct 2 16:22 original.txt
12573 lrwxrwxrwx 1 root root 12 Oct 2 16:22 symlink.txt -> original.txt
root@Shamikahasini:~# find . -inum $(ls -li original.txt | awk '{print $1}')
./hardlink.txt
./original.txt
root@Shamikahasini:~# find . -type l -ls
12573 0 lrwxrwxrwx 1 root root 12 Oct 2 16:22 ./symlink.txt -> original.txt
root@Shamikahasini:~# cat original.txt
cat hardlink.txt
cat symlink.txt
This is an inode test file.
This is an inode test file.
This is an inode test file.
root@Shamikahasini:~# echo "Modified through original file." >> original.txt
root@Shamikahasini:~# cat hardlink.txt
cat symlink.txt
```

Observation:

The `ls -li` and `stat` commands were used to examine inode numbers and file metadata. The hard link shared the same inode number as the original file, while the symbolic link had a separate inode containing a reference to the original filename. Creating a hard link increases the inode's link count but does not create a new inode, whereas creating a symbolic link allocates a separate inode.

Task-2

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and complexity.

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <time.h>
#include <string.h>

#define BUFFER_SIZE 4096

void traditional_copy(const char *src, const char *dest)
{
    int fd1, fd2;
    char buffer[BUFFER_SIZE];
    ssize_t n;

    fd1 = open(src, O_RDONLY);
    fd2 = open(dest, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd1 < 0 || fd2 < 0)
    {
        perror("open");
        exit(1);
    }

    while ((n = read(fd1, buffer, BUFFER_SIZE)) > 0)
    {
        write(fd2, buffer, n);
    }

    close(fd1);
    close(fd2);
}

void mmap_copy(const char *src, const char *dest)
{
    int fd1, fd2;
    struct stat st;
    char *src_map, *dest_map;
    double traditional_time, mmap_time;

    printf("FILE I/O PERFORMANCE COMPARISON\n");
    printf("-----\n");

    /* Traditional read/write */
    start = clock();

    traditional_copy(source, traditional);

    end = clock();

    traditional_time =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Traditional read/write completed.\n");
    printf("Time: %f seconds\n\n", traditional_time);

    /* Memory mapped I/O */
    start = clock();

    mmap_copy(source, mapped);

    end = clock();

    mmap_time =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Memory-mapped I/O completed.\n");
    printf("Time: %f seconds\n\n", mmap_time);

    printf("COMPARISON\n");
    printf("-----\n");

    printf("Traditional I/O : %f seconds\n", traditional_time);
    printf("mmap() I/O      : %f seconds\n", mmap_time);

    return 0;
}
```

```

root@Shamikahasini:~# nano mmap_compare.c
root@Shamikahasini:~# gcc mmap_compare.c -o mmap_compare
root@Shamikahasini:~# ./mmap_compare
FILE I/O PERFORMANCE COMPARISON
-----
Traditional read/write completed.
Time: 0.000155 seconds

Memory-mapped I/O completed.
Time: 0.000244 seconds

COMPARISON
-----
Traditional I/O : 0.000155 seconds
mmap() I/O      : 0.000244 seconds
root@Shamikahasini:~# ls -lh input.txt traditional.txt mmap_copy.txt
-rw-r--r-- 1 root root 56 Oct  2 16:14 input.txt
-rw-r--r-- 1 root root 56 Oct  2 16:26 mmap_copy.txt
-rw-r--r-- 1 root root 56 Oct  2 16:26 traditional.txt
root@Shamikahasini:~# cmp input.txt traditional.txt
cmp input.txt mmap_copy.txt
root@Shamikahasini:~# |

```

Observation:

The program successfully performed file copying using both traditional read()/write() operations and memory-mapped I/O using mmap(). Memory mapping provides direct access to file contents through memory addresses and can reduce explicit I/O operations. Traditional I/O is simpler to implement, while mmap() is more complex but can be efficient for large files and repeated or random access.